Draw It or Lose It

CS 230 Project Software Design Template

Version 1.0

Table of Contents

Document Revision History

Version	Date	Author	Comments
1.0	05/24/2026	Mickalia Reid	This revision includes an evaluation of different operating systems to be used to develop the game application and a list of recommendations for the development of the game application.

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The Gaming Room has requested the development of a web-based multiplayer game called Draw It or Lose It, where teams with multiple players compete to guess the puzzle using images of stock drawings as clues. The software must support multiple players and teams where unique game and team names are enforced and ensuring only one game instance exists in memory at any given time using unique identifiers for each instance.

Requirements

The software requirements for this application are as follows:

- The game should support multiplayer with multiple teams and players with the ability of each team being able to add multiple players
- Game and team names must be unique with the ability for players to check for an existing name
- The use of unique identifiers for each instance of game, player and team to ensure only one instance of the game exists in memory at a time.

Design Constraints

A few design constraints for web-based game development along with their implications are as follows:

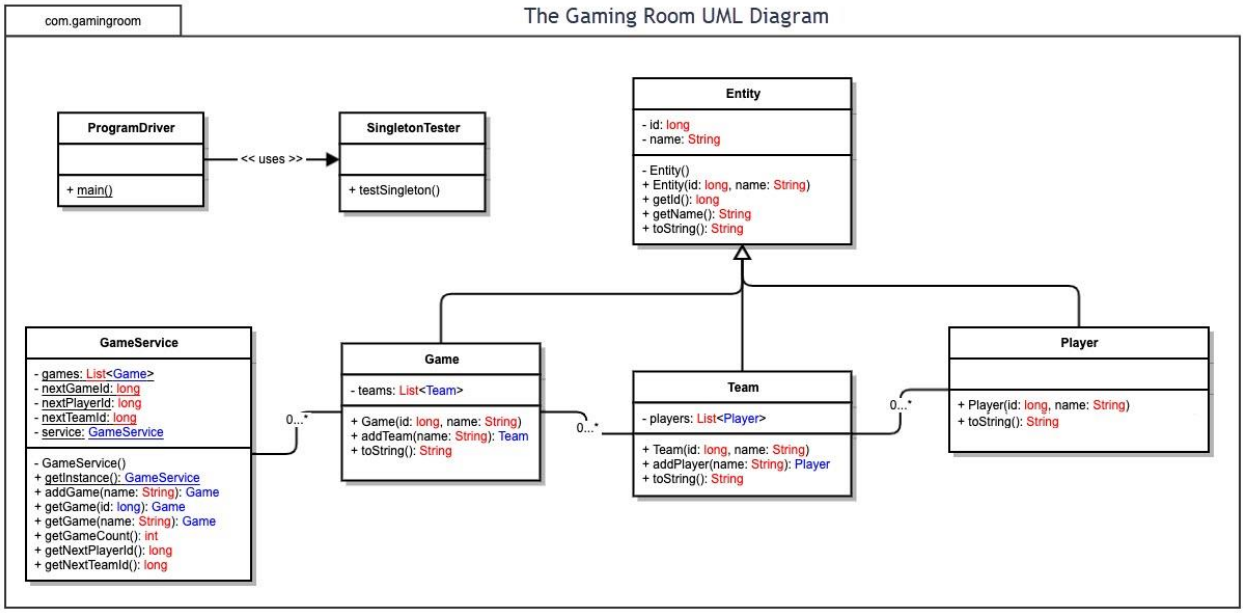
- **The game application must support multiple players and teams:**
  The implication of this is that the system will need to be highly responsive, compatible and scalable with different operating systems and internet speeds to maintain a smooth performance across different devices and browsers while supporting the multiplayer feature.
- **Only one instance of a game can exist in memory at a given time**:
  The implication of this is that the software will need to implement unique identifiers during the development to prevent duplication of games, teams or players from being created. The software will also need to utilize proper memory management to store each instance.
- **The game application must support the creation of unique games and team names**:
  The implication of this is that the software will need to include validation and checks during development to ensure that the game, team and player names are not duplicate. This will also foster organization and prevent user confusion.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The UML diagram below shows how the various classes work together to support the web-based game application. The *GameService* class manages the overall system of the game application through the creation and storing of different *Game* objects such as the *Team a*nd *Player* objects. The UML diagram shows that each *Game* contains multiple *Team* objects and each *Team* contains multiple *Player* objects. The *Entity* class acts as the parent class for the *Game, Team a*nd *Player* objects where these objects inherit various attributes such as *id* and *name*. The *ProgramDriver* and *SingletonTesteter* are used to test, create searches and run the game application.
The UML diagram demonstrates various object-oriented programming principles such as Inheritance, Encapsulation and Abstraction. Inheritance is demonstrated through the *Entity* class where the *Game, Team a*nd *Player* object inherit common attributes. This helps to prevent code duplication throughout the application. Encapsulation is demonstrated using the get method such as *getId().* This allows the control of access of private data such as the ID and names. Abstraction is demonstrated through the *Game, Team a*nd *Player* classes where only the important attributes, behaviors and methods needed for the application are included while details on internal implementation are hidden from users. This fosters simplicity and manageability.
The Singleton design pattern meets the client's requirements by ensuring only one instance of the game exists in memory at a time.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac OS are user friendly, secure and stable to host web-based applications. They strive in developmental environments and have the necessary functions to support web-based programming tools. However, Macs are expensive and less commonly used for large-scale web applications which might increase the cost for client.	Linux OS is highly reliable, secure and widely used for large-scale web applications. It performs well under heavy user traffic, supports scalability for multiplayer features and overall, it is more cost effective for clients with low/no licensing cost. However, Linux will require more technical expertise to maintain and configure the application.	Windows OS are user friendly and easy to integrate with Microsoft applications and developmental applications. However, Windows server requires licensing and more system resources which will increase the operational cost of the client.	Mobile Devices are not commonly used for large-scale web applications because they lack scalability, reliability and processing power. However, they provide convenience and better accessibility from any location which will increase user engagement. They also have various limitations such as smaller screens, different hardware performance and compatibility which will require additional testing during development and the creation of a more responsive and universal game design. The licensing costs are minimal.

Client Side	Development and testing of the game application will be more expensive for Mac as Apple devices are expensive and specialized hardware and expertise may be needed for development. Additional testing will also be required on the different web browsers to ensure compatibility.	Development and testing of the game application will require more technical expertise on Linux. Additional testing on different browsers and Linux distributions will be required to ensure compatibility and consistent functionality.	Development costs on Windows will be moderate as additional expertise, and training will not be needed as most developers are already familiar with Windows environment. Additional testing will be needed to ensure consistent performance and compatibility across the different Windows browsers.	Additional developmental time and expertise will be needed to create a responsive game design and optimizing game application for different screens sizes, OS and touch controls. Developmental costs may increase because additional tests will be needed for compatibility across many different devices, browsers and mobile platforms.

Development Tools	The relevant programming languages for Mac include Java, JavaScript, HTML and CSS. The tools such as IDEs used for Mac include Visual Studio Code, IDEA and Xcode. These may be used for the development and deployment of the web-based game application. Publishing in the Apple App Store will require yearly enrollment in the Apple Developer Program which will add to the developmental cost.	The relevant programming languages for Linux include Java, Python, JavaScript, HTML and CSS. The tools such as IDEs used for Linux include Eclipse, Visual Studio code, GIT and Apache. These may be used for the development and deployment of the web-based game application. These tools are also free to use.	The relevant programming languages for Windows include Java, C#, JavaScript, HTML and CSS. The tools such as IDEs used for Windows include Eclipse and Visual Studio Code. These may be used for the development and deployment of the web-based game application. Visual Studio code is free to use but pro/ enterprise versions will require a licensing fee.	The relevant programming languages for mobile devices include HTML5, JavaScript and CSS. The tools used for mobile devices include Android Studio and Xcode. These ensure compatibility across IOS and androids during the development and deployment of the web-based game application. Although the development tools are free, a one-time developer registration fee on Google Play for Android will be required. Development on IOS will require Apple Developer Program membership to be paid annually and apple hardware will be required to purchase for development.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: Linux is the recommended operating platform for the Draw It or Lose It game software because Linux is more cost-effective, support scalability, reliable, secure and widely used for web-based multiplayer software applications. For expansion across multiple platforms, Linux supports the increasing number of users that will be using the software, cloud deployment and virtualization without a high licensing cost.

2. **Operating Systems Architectures**: Linux platform uses a client-server architecture where the server hosts the game application and clients (end users) can connect through various web browsers and mobile devices. Linux's operating system services memory management, file system operations and process management of multiple game instances. The Linux platform architecture supports multiplayer gaming, cloud computing and virtualization which provides flexibility and higher performance.

3. **Storage Management**: MySQL would be an appropriate storage management system for the Draw It or Lose It game software. MySQL provides efficient data organization, quick data retrieval for searches (such as user accounts, scores and game sessions), supports multiple users and has reliable storage. MySQL also supports indexing, backup and recovery features that ensure quick access to the game data.

4. **Memory Management**: Linux uses memory management techniques such as resource allocation, virtual memory and caching to improve the game application performance. These techniques allow the game software to efficiently manage multiple users and game sessions without using excessive memory. These techniques also support the client's requirements of having only one instance of the game existing in memory at a time. The memory management of Linux also helps to maintain consistent application performance and stability, especially during high user activity.

5. **Distributed Systems and Networks**: The game application can use distributed software and network communications to communicate between various platforms such as web servers, cloud hosting and internet protocols allow communication between the clients and servers. Unreliable networks can affect network communications as unreliable networks can cause outages and poor connectivity which can interrupt gameplay and affect the user experience. A cloud-hosted database and load balancing can improve reliability and consistency while minimizing downtime of the application.

6. **Security**: To protect the user information on and between various platforms, Linux provides security features such as encryption, firewall protection, user authentication, access control permissions and regular security updates. Additional security features such as HTTPS should be used to further protect the application unauthorized access and cyber threats so users will feel more at ease and secure using the game application. Role-based access controls should also be implemented to protect sensitive user information and prevent unauthorized access to data.